

Git 快速使用手册

1. Git 的下载与配置

下载与安装

- **Windows:** 可以直接去 Git 官网下载安装程序并一路点击“下一步”安装。
- **macOS:** 打开终端（Terminal），输入 `Git --version`，如果没安装系统会自动弹窗提示安装。如果你使用 Homebrew，也可以输入 `brew install Git` 进行安装。
- **Linux (Ubuntu/Debian):** 打开终端，依次输入 `sudo apt-get update` 和 `sudo apt-get install Git-all` 即可完成安装。

初始配置（必做）

安装完成后，你需要告诉 Git 你是谁。打开命令行终端（Windows 用户可以打开 Git Bash），输入以下两行命令设置你的名字和邮箱（只需设置这一次即可）：

```
Git config --global user.name "你的名字拼音或英文名"
```

```
Git config --global user.email "你的邮箱地址"
```

这两个信息会出现在你未来的每一次代码提交记录中。

2. 本地基础操作命令

新建工程 (Git init)

当你在本地新建了一个项目（比如刚刚用 STM32CubeIDE 或 Keil 建好了一个工程文件夹），你想让 Git 开始管理它。

操作：在终端中进入你的项目文件夹，然后输入：

```
Git init
```

这会在当前目录生成一个隐藏的 .Git 文件夹，代表 Git 已经开始接管这个项目了。

提交代码 (Git add 与 Git commit)

当你完成了一个小功能（比如写好了定时器中断），你需要把它“存”下来，形成一个历史记录。

步骤 1：把修改的文件添加到暂存区（告诉 Git 这些文件我要提交了）：

```
Git add.
```

(注意: `add` 后面有一个空格和一个英文句号., 意思是把当前目录下所有修改过的文件都加进去)

步骤 2: 给这次提交写一句说明, 正式存入历史记录:

```
Git commit -m "完成了定时器中断的初始化配置"
```

(注意: `-m` 后面双引号里的文字就是你对这次修改的描述, 一定要写清楚)

3. 在 GitHub/GitLab 上新建并公开项目

当你完成本地代码的编写后, 通常会希望把它传到云端 (如 GitHub 或 GitLab) 保存, 或者作为开源项目展示在你的简历中。

步骤 1: 在网页端创建云端仓库

- **GitHub:** 登录 GitHub 网页, 点击页面右上角的 + 号图标, 选择 **"New repository"**。填写你的项目名称 (Repository name)。
- **GitLab:** 登录 GitLab, 在顶部导航栏找到 + 号图标, 选择新建项目 (New project) 或上传文件, 填写项目名称。

步骤 2: 设置项目为公开 (Public)

- **创建时设置:** 在新建仓库的页面中, 找到“可见性 (Visibility Level 或 Repository visibility)”选项, 直接勾选 **Public** (公开), 这样任何人都可以看到你的代码。
- **创建后修改:** 如果你之前建立的是私有项目 (**Private**), 想要改成公开, 可以进入该项目的网页, 点击顶部的 **Settings** (设置), 滑动到页面最底部的 **Danger Zone** (危险区域), 找到 **"Change repository visibility"**, 将其修改为 **Public** 即可。

步骤 3: 将本地代码与云端项目关联并推送

云端仓库建好后, 网页会给你提供一个以 `.Git` 结尾的链接 (例如 `https://Github.com/你的名字/你的项目.Git`)。回到你本地电脑的项目终端, 依次输入以下命令:

1. 关联云端仓库: 告诉本地 Git 你的云端仓库地址在哪里:

```
Git remote add origin https://Github.com/你的名字/你的项目.Git
```

(注意: `origin` 是给云端仓库起的一个默认本地别名)

2. 首次推送代码到云端: 把你本地已经 `commit` 好的代码推送到云端:

```
Git push -u origin main
```

(注意：如果你本地的默认分支名叫 *master*，则把命令里的 *main* 换成 *master*。-u 参数的作用是让 *Git* 记住这个目标，以后更新代码只需要简单输入 *Git push* 即可)

4. 其他日常云端操作

日常同步 (Git push 与 Git pull)

在完成了上面的首次关联后，以后的日常开发同步就非常简单了：

- 把本地更新传到网上：每次完成新的 *Git commit* 后，直接输入 *Git push*。
- 把网上的更新拉到本地：如果同学修改了代码并传到了网上，你在终端输入 *Git pull* 就能把最新的代码拉取到本地。

克隆别人的工程 (Git clone)

如果你想下载 *GitHub* 上开源的代码，或者老师发给你的参考工程。

操作：在终端中输入命令加上工程的链接：

```
Git clone https://Github.com/xxxxx/xxxxx.Git
```

这会把网上的整个项目完完整整地下载到你的电脑上。

将自己的 *Git* 项目链接添加到每周提交的研讨记录中

(注意：更多的操作可以自行在网络平台上学习，例如 *bilibili* 等等平台，这里的操作只是一个简单的介绍。)